The internet we were promised
July 16. An important update for readers in the United States.

You deserve an explanation, so please don't skip this 1-minute read. It's Wednesday, July 16, and our fundraiser won't last long. If you've lost count of how many times you've visited Wikipedia this year, we hope that means it's given you at least $2.75 of knowledge. Please join the 2% of readers who give to keep this valuable resource ad-free and available for all.

After nearly 25 years, Wikipedia is still the internet we were promised—an oasis of free and collaborative knowledge. By visiting Wikipedia today, you're choosing a free and fair internet: a space to find facts without the distraction of ads or agendas of wealthy owners.

Most readers donate because Wikipedia is useful to them, others because Wikipedia is more important than ever. If you feel the same, please donate $2.75 now—or consider a monthly gift to help all year. Thank you.

WIKIMEDIA FOUNDATION Proud host of Wikipedia and its sister sites

Problems donating? | frequently asked questions | Other ways to give | I already donated

We never sell your information. By submitting, you are agreeing to our donor privacy policy. The Wikimedia Foundation is a nonprofit, tax-exempt organization. If you make a recurring donation, you will be debited by the Wikimedia Foundation until you notify us to stop. We'll send you an email which will include a link to easy cancellation instructions.

1. How often would you like to donate?
Once | Monthly | Annual
Support Wikipedia year-round
2. Please select an amount (USD)
The average donation in the United States is around $13.
$2.75 | $15 | $25
$50 | $100 | $250
$500 | Other
3. Please select a payment method
Debit or Credit Card | Online Banking | venmo | Apple Pay | G Pay | PayPal
Continue
Maybe later

fork (system call)

From Wikipedia, the free encyclopedia

In computing, particularly in the context of the Unix operating system and its workalikes, **fork** is an operation whereby a process creates a copy of itself. It is an interface which is required for compliance with the POSIX and Single UNIX Specification standards. It is usually implemented as a C standard library wrapper to the fork, clone, or other system calls of the kernel. Fork is the primary method of process creation on Unix-like operating systems.

Overview [edit]

In multitasking operating systems, processes (running programs) need a way to create new processes, e.g. to run other programs. Fork and its variants are typically the only way of doing so in Unix-like systems. For a process to start the execution of a different program, it first forks to create a copy of itself. Then, the copy, called the "child process", makes any environment changes the child will need and then calls the exec system call to overlay itself with the new program: it ceases execution of its former program in favor of the new. (Or, in rarer cases, the child forgoes the exec and continues executing, as a separate process, some other functionality of the original program.)

The fork operation creates a separate address space for the child. The child process has an exact copy of all the memory segments of the parent process. In modern UNIX variants that follow the virtual memory model from SunOS-4.0, copy-on-write semantics are implemented and the physical memory need not be actually copied. Instead, virtual memory pages in both processes may refer to the same pages of physical memory until one of them writes to such a page: then it is copied. This optimization is important in the common case where fork is used in conjunction with exec to execute a new program: typically, the child process performs only a small set of actions before it ceases execution of its program in favour of the program to be started, and it requires very few, if any, of its parent's data structures.

When a process calls fork, it is deemed the parent process and the newly created process is its child. After the fork, both processes not only run the same program, but they resume execution as though both had called the system call. They can then inspect the call's return value to determine their status, child or parent, and act accordingly.

The history of fork dates back to the 1960s as one of the earliest references to a fork concept appeared in *A Multiprocessor System Design* by Melvin Conway, published in 1962.[1] Conway's paper motivated the implementation by L. Peter Deutsch of fork in the GENIE time-sharing system, where the concept was borrowed by Ken Thompson for its earliest appearance[2] in Research Unix.[3][4] Fork later became a standard interface in POSIX.[5]

Technical [edit]

Example [edit]

The following variant of the "Hello, World!" program demonstrates the mechanics of the fork system call in the C programming language. The program forks into two processes, each deciding what functionality they perform based on the return value of the fork system call. Boilerplate code such as header inclusions has been omitted.

```
int main(void)
{
    pid_t pid = fork();

    if (pid == -1) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }
    else if (pid == 0) {
        printf("Hello from the child process!\n");
        _exit(EXIT_SUCCESS);
    }
    else {
        int status;
        (void)waitpid(pid, &status, 0);
    }
    return EXIT_SUCCESS;
}
```

What follows is a dissection of this program.

```
pid_t pid = fork();
```

The first statement in `main` calls the `fork` system call to split execution into two processes. The return value of `fork` is recorded in a variable of type `pid_t`, which is the POSIX type for process identifiers (PIDs).

```
    if (pid == -1) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }
```

Minus one indicates an error in `fork`: no new process was created, so an error message is printed.

If `fork` was successful, then there are now two processes, both executing the `main` function from the point where `fork` has returned. To make the processes perform different tasks, the program must branch on the return value of `fork` to determine whether it is executing as the *child* process or the *parent* process.

```
    else if (pid == 0) {
        printf("Hello from the child process!\n");
        _exit(EXIT_SUCCESS);
    }
```

In the child process, the return value appears as zero (which is an invalid process identifier). The child process prints the desired greeting message, then exits. (For technical reasons, the POSIX `_exit` function must be used here instead of the C standard `exit` function.)

```
    else {
        int status;
        (void)waitpid(pid, &status, 0);
    }
```

The other process, the parent, receives from `fork` the process identifier of the child, which is always a positive number. The parent process passes this identifier to the `waitpid` system call to suspend execution until the child has exited. When this has happened, the parent resumes execution and exits by means of the `return` statement.

Communication [edit]

The child process starts off with a copy of its parent's file descriptors.[6] For inter-process communication with data exchange and synchronization between parent and child processes, pipes can be used.[6] The parent process will often create one or several pipes, and then after forking the processes will close the ends of the pipes that they do not need.[6] To share the same variable between the parent and child processes, memory mapping or shared memory must be used.[7]

Variants [edit]

Vfork [edit]

Vfork is a variant of fork with the same calling convention and much the same semantics, but only to be used in restricted situations. It originated in the 3BSD version of Unix,[8][9][10] the first Unix to support virtual memory. It was standardized by POSIX, which permitted vfork to have exactly the same behavior as fork, but was marked obsolescent in the 2004 edition[11] and was replaced by posix_spawn() (which is typically implemented via vfork) in subsequent editions.

When a vfork system call is issued, the parent process will be suspended until the child process has either completed execution or been replaced with a new executable image via one of the "exec" family of system calls. The child borrows the memory management unit setup from the parent and memory pages are shared among the parent and child process with no copying done, and in particular with no copy-on-write semantics,[11] hence, if the child process makes a modification in any of the shared pages, no new page will be created and the modified pages are visible to the parent process too. Since there is absolutely no page copying involved (consuming additional memory), this technique is an optimization over plain fork in full-copy environments when used with exec. In POSIX, using vfork for any purpose except as a prelude to an immediate call to a function from the exec family (and a select few other operations) gives rise to undefined behavior.[11] As with vfork, the child borrows data structures rather than copying them. vfork is still faster than a fork that uses copy on write semantics.

System V did not support this function call before System VR4 was introduced,[citation needed] because the memory sharing that it causes is error-prone:

> *Vfork* does not copy page tables so it is faster than the System V *fork* implementation. But the child process executes in the same physical address space as the parent process (until an exec or exit) and can thus overwrite the parent's data and stack. A dangerous situation could arise if a programmer uses *vfork* incorrectly, so the onus for calling *vfork* lies with the programmer. The difference between the System V approach and the BSD approach is philosophical: Should the kernel hide idiosyncrasies of its implementation from users, or should it allow sophisticated users the opportunity to take advantage of the implementation to do a logical function more efficiently?
>
> —Maurice J. Bach[12]

Similarly, the Linux man page for vfork strongly discourages its use.[citation needed][failed verification – see discussion]

> It is rather unfortunate that Linux revived this specter from the past. The BSD man page states: "This system call will be eliminated when proper system sharing mechanisms are implemented. Users should not depend on the memory sharing semantics of vfork() as it will, in that case, be made synonymous to fork(2)."

Other problems with vfork include deadlocks that might occur in multithreaded programs due to interactions with dynamic linking.[13] As a replacement for the vfork interface, POSIX introduced the posix_spawn family of functions that combine the actions of fork and exec. These functions may be implemented as library routines in terms of fork, as is done in Linux,[14] or in terms of vfork for better performance, as is done in Solaris,[13][14] but the POSIX specification notes that they were "designed as kernel operations", especially for operating systems running on constrained hardware and real-time systems.[15]

While the 4.4BSD implementation got rid of the vfork implementation, causing vfork to have the same behavior as fork, it was later reinstated in the NetBSD operating system for performance reasons.[9]

Some embedded operating systems such as uClinux omit fork and only implement vfork, because they need to operate on devices where copy-on-write is impossible to implement due to lack of a memory management unit.

Rfork [edit]

The Plan 9 operating system, created by the designers of Unix, includes fork as a variant of a new function called "rfork" that permits fine-grained sharing of resources between parent and child processes, including the address space (except for a stack segment, which is unique to each process), environment variables and the filesystem namespace;[16] this makes it a unified interface for the creation of both processes and threads within them.[17] Both FreeBSD[18] and IRIX adopted the rfork system call from Plan 9, the latter renaming it "sproc".[19]

Clone [edit]

`clone` is a system call in the Linux kernel that creates a child process that may share parts of its execution context with the parent. Like FreeBSD's rfork and IRIX's sproc, Linux's clone was inspired by Plan 9's rfork and can be used to implement threads (though application programmers will typically use a higher-level interface such as pthreads, implemented on top of clone). The "separate stacks" feature from Plan 9 and IRIX has been omitted because (according to Linus Torvalds) it causes too much overhead.[18]

Forking in other operating systems [edit]

This section **relies largely or entirely on a single source**. Relevant discussion may be found on the talk page. Please help improve this article by introducing citations to additional sources.
Find sources: "Fork" system call – news · newspapers · books · scholar · JSTOR *(February 2015)*

In the original design of the VMS operating system (1977), a copy operation with subsequent mutation of the content of a few specific addresses for the new process as in forking was considered risky.[citation needed] Errors in the current process state may be copied to a child process. Here, the metaphor of process spawning is used: each component of the memory layout of the new process is newly constructed from scratch. The spawn metaphor was later adopted in Microsoft operating systems (1993).

The POSIX-compatibility component of VM/CMS (OpenExtensions) provides a very limited implementation of fork, in which the parent is suspended while the child executes, and the child and the parent share the same address space.[20] This is essentially a vfork labelled as a fork. (This applies to the CMS guest operating system only; other VM guest operating systems, such as Linux, provide standard fork functionality.)

See also [edit]

- Fork bomb
- Fork–exec
- exit (system call)
- spawn (computing)
- wait (system call)

References [edit]

1. ^ Nyman, Linus (25 August 2016). "Notes on the History of Fork and Join". *IEEE Annals of the History of Computing*. **38** (3): 84–87. doi:10.1109/MAHC.2016.34.
2. ^ "5.3.s from Research UNIX". *GitHub*. 1970.
3. ^ Ken Thompson and Dennis Ritchie (3 November 1971). "SYS FORK (II)" (PDF). *UNIX Programmer's Manual*. Bell Laboratories.
4. ^ Ritchie, Dennis M.; Thompson, Ken (July 1978). "The UNIX Time-Sharing System" (PDF). *Bell System Tech. J.* **57** (6). AT&T: 1905–1929. doi:10.1002/j.1538-7305.1978.tb02136.x. Retrieved 22 April 2014.
5. ^ a b c `fork` – System Interfaces Reference, The Single UNIX Specification, Version 5 from The Open Group
6. ^ a b c `pipe` – System Interfaces Reference, The Single UNIX Specification, Version 5 from The Open Group
7. ^ "shmat()". *Linux man page*. Retrieved 2025-07-12.
8. ^ a b c `vfork(2)` – Linux Programmer's Manual – System Calls
9. ^ a b "NetBSD Documentation: Why implement traditional vfork()". *NetBSD Project*. Retrieved 16 October 2013.
10. ^ "*vfork(2)*". *UNIX Programmer's Manual, Virtual VAX-11 Version*. University of California, Berkeley. December 1979.
11. ^ a b c d e `vfork` – System Interfaces Reference, The Single UNIX Specification, Version 3 from The Open Group
12. ^ Bach, Maurice J. (1986). *The Design of The UNIX Operating System*. Prentice-Hall. pp. 291–292. Bibcode:1986design.book....B.
13. ^ a b c d Nakhimovsky, Greg (May 2006). "Minimizing Memory Usage for Creating Application Subprocesses". *Oracle Technology Network*. Oracle Corporation. Archived from the original on Sep 22, 2019.
14. ^ The OpenSolaris posix_spawn() implementation
15. ^ `posix_spawn` – System Interfaces Reference, The Single UNIX Specification, Version 5 from The Open Group
16. ^ `fork(2)` – Plan 9 Programmer's Manual, Volume 1
17. ^ `intro(2)` – Plan 9 Programmer's Manual, Volume 1
18. ^ `rfork(2)` – FreeBSD System Calls Manual
19. ^ a b Torvalds, Linus (1999). "The Linux edge". *Communications of the ACM*. *Open Sources: Voices from the Open Source Revolution*. O'Reilly. ISBN 978-1-56592-582-3.
20. ^ "z/VM > z/VM 6.2.0 > Application Programming > z/VM V6R2 OpenExtensions POSIX Conformance Document > POSIX.1 Conformance Document > Section 3. Process Primitives > 3.1 Process Creation and Execution > 3.1.1 Process Creation". IBM. Retrieved April 21, 2015.

Categories: Process (computing) | C POSIX library | System calls

This page was last edited on 12 July 2025, at 10:29 (UTC).
Text is available under the Creative Commons Attribution-ShareAlike 4.0 License; additional terms may apply. By using this site, you agree to the Terms of Use and Privacy Policy. Wikipedia® is a registered trademark of the Wikimedia Foundation, Inc., a non-profit organization.
Privacy policy About Wikipedia Disclaimers Contact Wikipedia Code of Conduct Developers Statistics Cookie statement Mobile view